

Архитектурное описание системы

1. Общие сведения о системе

Разрабатываемая система представляет собой однопользовательскую игру в жанре roguelike с консольным интерфейсом. Игрок управляет персонажем с клавиатуры, перемещается по карте уровня, взаимодействует с мобами и предметами. Предметы могут находиться на карте, подбираться в инвентарь, а также надеваться и сниматься, надетые предметы влияют на характеристики персонажа. Игра состоит из последовательности уровней, уровни могут быть заранее определёнными (загружаться из набора уровней) или генерироваться процедурно. Игра завершается победой при прохождении всех уровней либо поражением при смерти персонажа.

2. Architectural drivers

При проектировании системы учитываются следующие факторы. Архитектура должна быть расширяемой, так как дальнейшие задания предполагают развитие механик и усложнение логики. Требуется высокая скорость разработки, компоненты должны быть слабо связаны, чтобы изменения в генерации уровня, данных или рендере не приводили к переработке всего приложения. Интерфейс должен работать без заметных задержек при вводе и перерисовке. Дополнительным ограничением является простота реализации и понятность структуры для учебного проекта.

3. Роли и случаи использования

В системе присутствует один основной актор - игрок. Игрок запускает приложение и взаимодействует с игрой через клавиатуру. Система отображает состояние уровня и результаты действий пользователя.

3.1. Прецедент П1. Игра в Roguelike игру

Основной исполнитель: игрок.

Заинтересованные лица и их требования: игрок хочет получить удовольствие от игры и проникнуться духом старых игр.

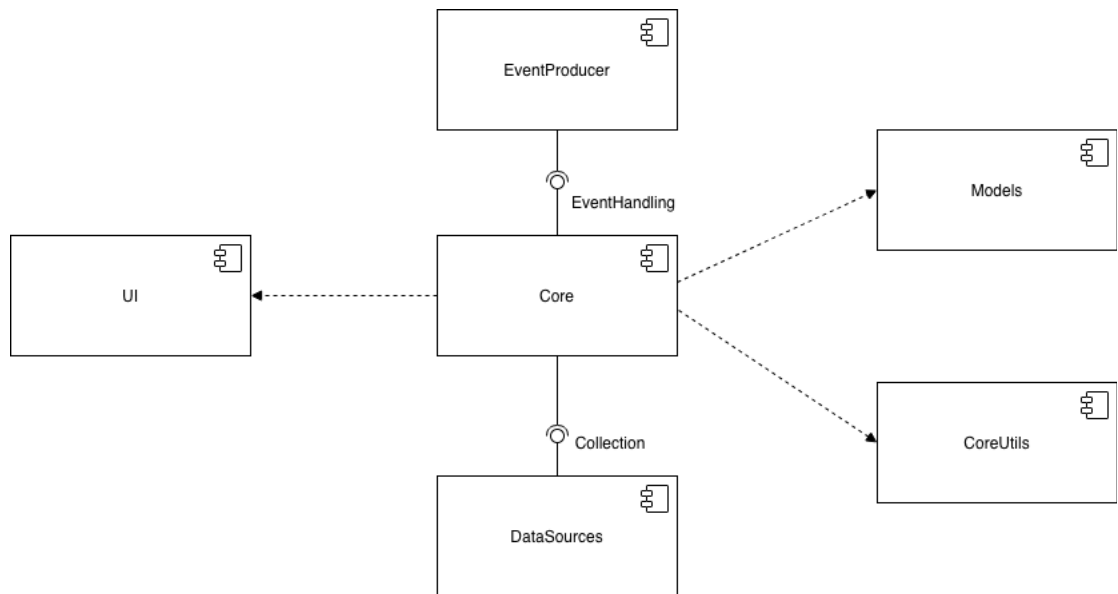
Основной успешный сценарий.

1. Игрок запускает игру.
 2. Система предлагает пользователю ввести имя персонажа, после чего пользователь вводит имя.
 3. Далее система предлагает пользователю выбрать пол, расу и класс персонажа - пользователь делает выбор.
 4. После завершения ввода параметров персонажа система генерирует новый уровень.
 5. Пользователь проходит уровень, уничтожая всех мобов.
 6. После выполнения условия завершения уровня система предлагает перейти на новый уровень.
 7. Пользователь подтверждает переход.
- Действия, описанные в пп. 5-7, повторяются, пока пользователь не пройдет все уровни.
8. После прохождения последнего уровня система поздравляет пользователя с прохождением игры.

Расширения: при закрытии пользователем терминала с игрой приложение завершается без выполнения дополнительных действий.

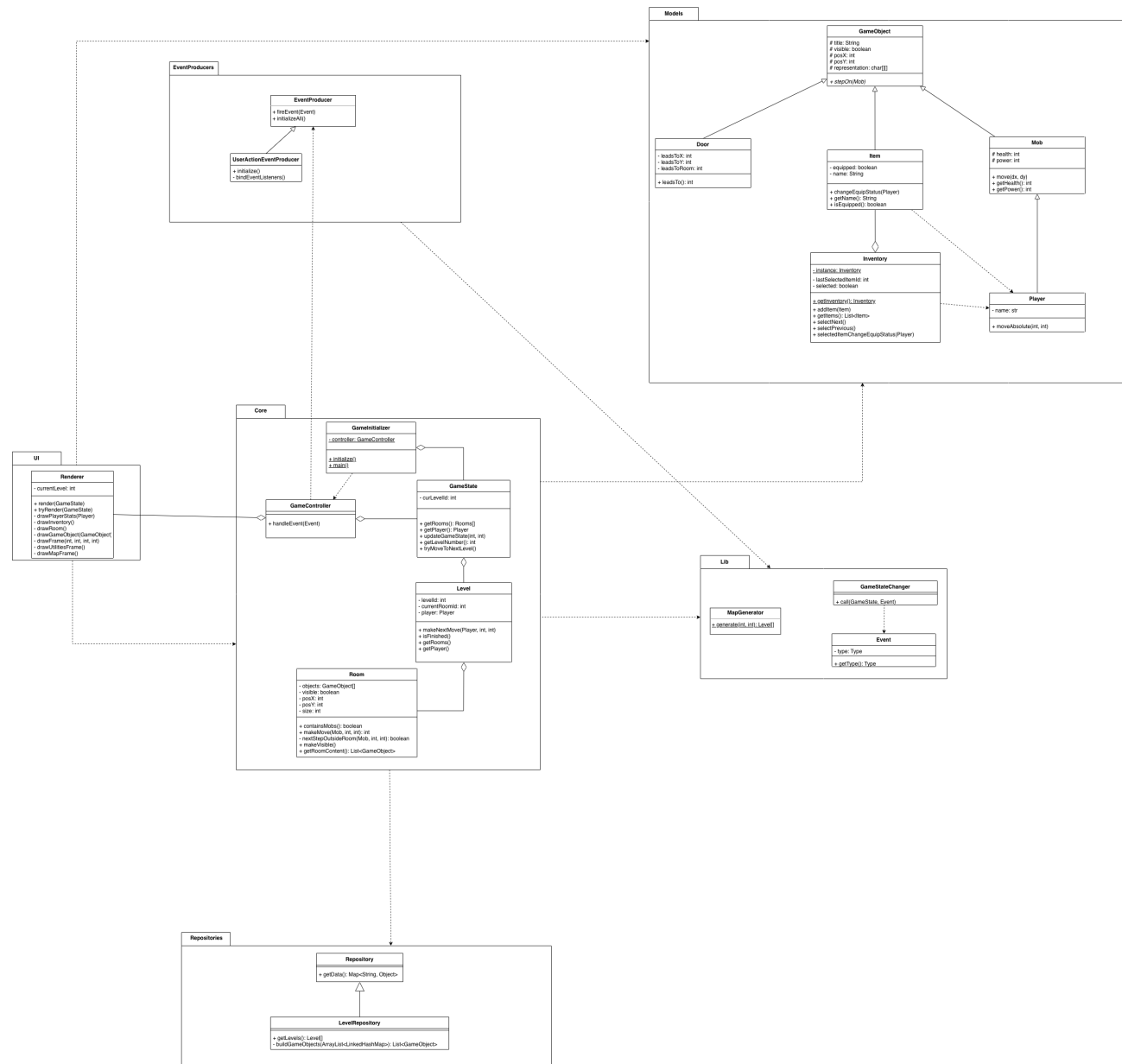
4. Диаграмма компонентов и описание компонентов

Система разделена на логические компоненты: подсистема формирования событий (ввод), ядро (игровая логика и состояние), пользовательский интерфейс (отрисовка), доменная модель (сущности игры), вспомогательные алгоритмы и репозитории данных. Поток управления в типичном цикле работы следующий: подсистема ввода формирует событие, ядро обрабатывает событие и изменяет состояние игры, после чего интерфейс перерисовывает актуальное состояние.



Подсистема формирования событий включает интерфейс EventProducer и его реализации UserActionEventProducer и PeriodicEventProducer, которые преобразуют пользовательские действия и периодические сигналы в события. Ядро включает контроллер обработки событий GameController, структуру состояния GameState и инициализатор GameInitializer. Интерфейс реализован компонентом UI с классом Renderer, который отвечает за отображение текущего состояния. Доменная модель (Models) содержит классы игровых объектов (GameObject, Door), сущности персонажа и противников (Player, Mob), а также предметы и инвентарь (Item, Inventory). Репозитории (Repositories) предоставляют доступ к предопределённым данным (например, LevelRepository), а также вспомогательные модули/утилиты могут использоваться для построения карт и объектов уровня (например, MapGenerator).

5. Диаграмма классов и логическая структура



Ключевым классом подсистемы Core является GameController, который обрабатывает входные события через метод `handleEvent(Event)`. Класс GameState хранит идентификатор текущего уровня (`curLevelId: int`), предоставляет доступ к игроку через `getPlayer(): Player`, к комнатам/содержимому уровня через `getRooms(): Rooms[]`, а также содержит операции изменения состояния `updateGameState(int, int)` и перехода на следующий уровень

tryMoveToNextLevel(). Инициализация выполняется через GameInitializer (initialize(), main()).

Модель уровня представлена классами Level и Room. Level хранит идентификатор уровня и текущей комнаты (levelId: int, currentRoomId: int), ссылку на игрока (player: Player) и предоставляет методы для выполнения хода и проверки завершения уровня (makeNextMove(...), isFinished(), getRooms(), getPlayer()). Room хранит набор объектов (objects: GameObject[]), видимость и геометрию комнаты (visible, posX/posY, size), а также операции доступа к содержимому и вспомогательные проверки (containsMobs(), getRoomContent(), makeVisible()).

В доменной модели базовым классом игровых объектов является GameObject (title, visible, posX/posY, representation). Объекты могут реагировать на наступление персонажа/моба через stepOn(Mob). От GameObject наследуется Door, задающая переходы между комнатами (leadsToX, leadsToY, leadsToRoom, leadsTo()).

Класс Mob содержит базовые боевые характеристики (health, power) и операции перемещения (move(dx, dy), getHealth(), getPower()). Класс Player является специализацией Mob и дополнительно хранит имя (name) и реализует абсолютное перемещение moveAbsolute(int, int).

Инвентарь представлен классом Inventory (singleton), содержащим список предметов и операции выбора/экипировки: addItem(Item), getItems(), selectNext(), selectPrevious(), selectedItemChangeEquipStatus(Player). Предмет описывается классом Item (equipped, name) и операциями changeEquipStatus(Player), getName(), isEquipped().

Взаимодействие между событием и изменением состояния формализовано через классы Event (type: Type, getType()) и GameStateChanger, который

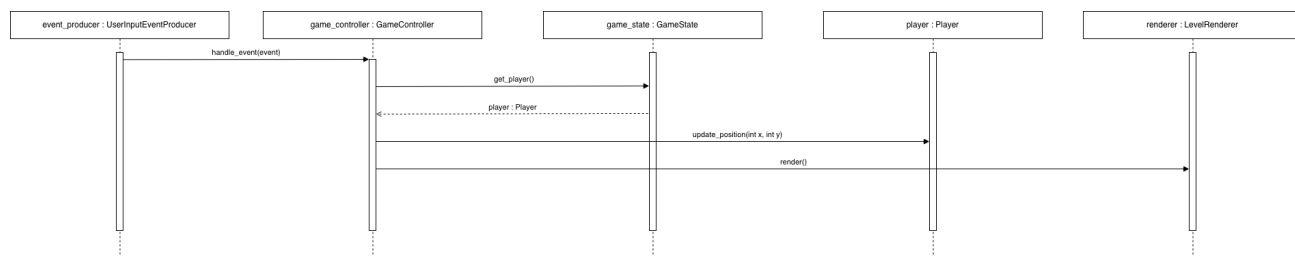
выполняет вызов `call(GameState, Event)` и тем самым сопоставляет тип события соответствующему изменению `GameState`.

Подсистема репозитория описывается интерфейсом `Repository (getData(): Map<String, Object>)` и конкретным классом `LevelRepository (getLevels(): Level[])`, который также содержит внутренние операции построения объектов уровня (`buildGameObjects(...)`).

6. Диаграммы последовательностей (взаимодействия)

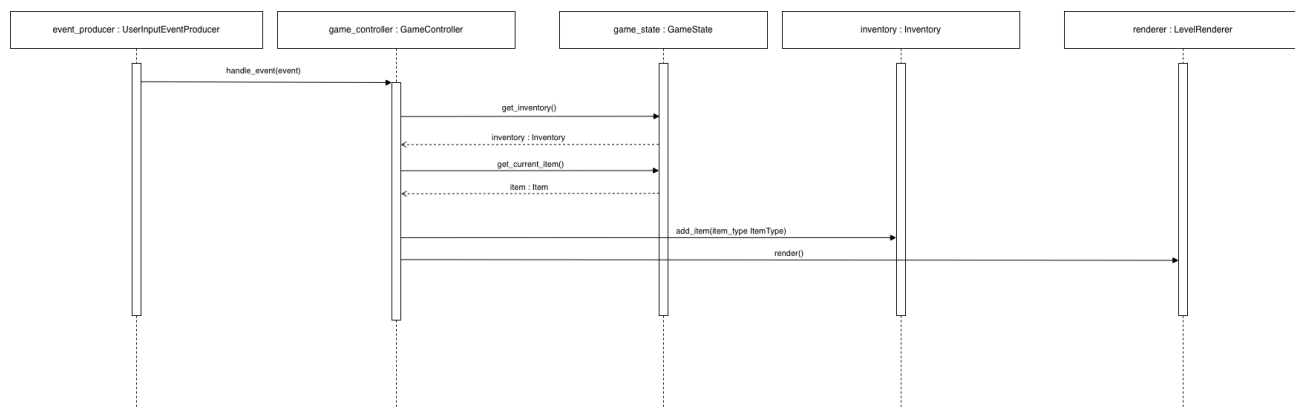
Диаграммы последовательностей фиксируют типовые сценарии работы системы и показывают передачу управления между объектами при обработке событий пользователя.

6.1. Перемещение персонажа



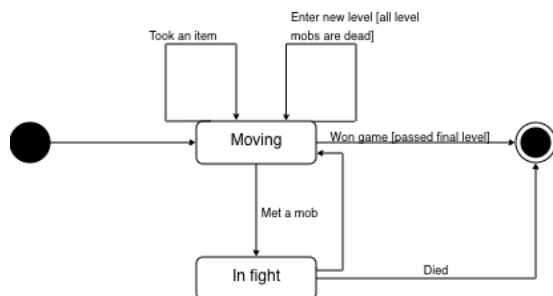
`UserActionEventProducer` формирует событие и инициирует его обработку в `GameController.handle_event(Event)`. `GameController` получает игрока через `GameState.get_player(): Player` и изменяет позицию игрока (например, через метод `Player.update_position(int x, int y)` или эквивалентный вызов). После изменения состояния `GameController` вызывает `Renderer.render(GameState)`.

6.2. Подбор предмета



`UserActionEventProducer` формирует событие подбора предмета и передаёт его в `GameController.handle_event(Event)`. `GameController` получает доступ к `Inventory` (через `GameState.inventory` или через аксессор) и добавляет предмет в инвентарь (например, вызовом `Inventory.add_item(...)`). Далее выполняется `Renderer.render(GameState)`.

7. Диаграмма состояний



Состояния игры представлены режимами `Moving` (исследование уровня), `In fight` (бой) и `Game over` (завершение игры). Переход `Moving > In fight` происходит при встрече с мобом. После боя возможен переход в `Moving` при победе или переход в `Game over` при смерти игрока. Переход на следующий уровень выполняется при выполнении условия завершения уровня и не меняет основной режим (остается `Moving`), но обновляет текущий `Level`. При прохождении последнего уровня выполняется переход в `Game over` с результатом “победа”.